# SIMATS ENGINEERING

## ASSESSMENT TOOL 2 : Scenario-Based

**COURSE NAME:** Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1514

---

**COURSE OUTCOME COVERED**

**CO2:** *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**

Question Count: 5

---

**Code of Conduct**

I **RASHMITHA SENTHIL KUMAR (192525034)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: **RASHMITHA .S**

**Assessment Tasks Scenario-Based Questions**

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.
   Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
   Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

**Scenario-Based Rubric**

| **Criteria** | **Weightage** | **Level 4 - Exemplary** | **Level 3 - Proficient** | **Level 2 - Developing** | **Level 1 - Beginning** |
|---|---|---|---|---|---|
| Understanding of Scenario | 20% | Demonstrates complete understanding of the scenario context | Good understanding with minor gaps | Partial understanding | Fails to understand the scenario |
| Identification of Risks / Issues | 20% | Identifies all major issues correctly | Identifies most issues | Limited issue identification | Misses major issues |
| Application of Concepts | 25% | Applies virtualization concepts effectively | Applies concepts with minor mistakes | Partially correct application | Weak application |
| Decision Justification | 20% | Decisions are well justified and technically strong | Reasonable justification | Weak justification | No useful justification |
| Clarity | 15% | Clear and well-organized response | Generally clear | Somewhat unclear | Poorly presented |

# QUESTION 1 – CLOUD VIDEO STREAMING ARCHITECTURE

Scenario: A media company is migrating a video streaming platform to the cloud and must support millions of concurrent viewers with minimal buffering. The solution must incorporate scalability, caching, load balancing, auto-scaling and cost optimization.

**Scenario interpretation and requirements:**

The primary challenge is to deliver video at high aggregate throughput while preserving playback continuity during popular live events and sudden audience growth. A streaming service includes an ingest and encoding pipeline, a control plane for authentication and playback manifests, and a media-delivery data plane. These components scale differently: control-plane APIs need low-latency request handling, while video segments generate most network egress. The architecture must also consider regional demand, device compatibility, start-up latency, content rights, security and the operational consequences of a cache miss.

**Proposed cloud architecture:**

Video files or live feeds enter an ingest layer, where transcoding workers generate multiple bitrate renditions. Packaging produces HLS or MPEG-DASH manifests and short media segments. Durable object storage holds origin assets, and a geographically distributed content delivery network (CDN) caches popular segments close to viewers. A global traffic manager routes API requests to healthy regional endpoints, while a regional application load balancer distributes them across stateless playback, authentication and catalog services. A managed database stores metadata and entitlements; a distributed cache accelerates read-heavy metadata queries. Telemetry from player devices and services informs scaling decisions and quality monitoring.

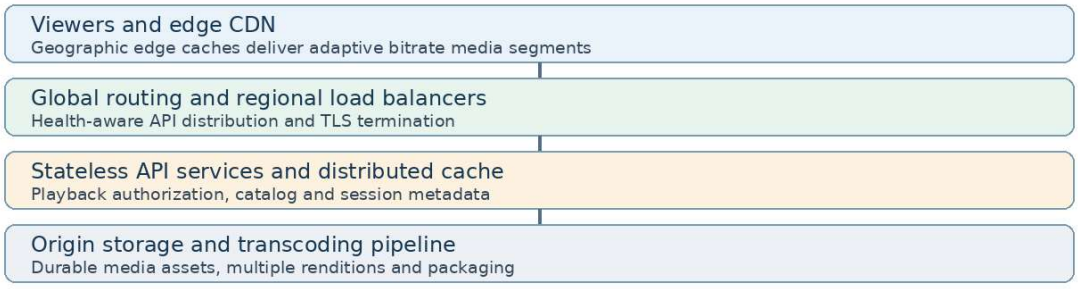



**Figure 1 – Recommended scalable streaming architecture with CDN-based media delivery.**

**Scalability and caching strategy:**

Horizontal scaling is preferable for stateless API services because new instances can be added without migrating session state. Frequently accessed metadata should use a managed in-memory cache with appropriate time-to-live values and explicit invalidation on updates. Video segments should be cached at CDN edges using versioned immutable URLs and carefully selected cache-control policies. A multi-tier cache may combine CDN edge, regional origin shield and object-storage origin to reduce repeated fetches. Cache hit ratio, origin requests and cache eviction rates should be measured continuously. Personalized or rights-sensitive responses must not be cached publicly unless cache keys and access policies correctly isolate users.

Adaptive bitrate streaming reduces buffering by allowing players to choose an appropriate rendition based on observed throughput and buffer health. It is not a substitute for adequate capacity, but it makes the experience resilient to network variability. For live streaming, segment duration, latency and cacheability must be balanced; very short segments can reduce delay but increase request overhead.

**Load balancing and auto-scaling:**

A global traffic manager should select healthy regions using latency-aware or policy-based routing, with a documented failover policy. Within each region, an application load balancer spreads API traffic across multiple availability zones and removes unhealthy instances from service. Stateless services should use health checks, connection draining and rolling deployments. Auto-scaling can react to request rate, CPU utilization, queue backlog and response latency, using minimum warm capacity and scheduled pre-scaling for anticipated events. Encoding workers should scale from ingestion queues rather than solely from CPU utilization. CDN delivery should absorb the bulk of segment traffic, avoiding an impractical design in which every viewer downloads video directly from application servers.

**Cost optimization and operational risks:**

Network egress and CDN transfer charges may dominate streaming cost, so the first optimization is a high cache-hit ratio and appropriate bitrate ladder. Additional savings come from object-storage lifecycle tiers, encoding once for reusable renditions, event-driven transcoding, reserved or committed capacity for steady demand, and lower-cost interruptible compute only for retryable jobs. Budgets, unit-cost dashboards and per-stream cost attribution should prevent hidden cost growth.

Important risks include regional outage, CDN misconfiguration, origin overload, encoding backlog, token leakage, DDoS attacks and licensing violations. Multi-CDN arrangements may reduce dependence on one provider but introduce routing and contractual complexity.

**Table 1. Cloud Video Streaming Architecture Design Decisions and Validation Metrics**

| Design decision | Technical justification | Validation metric |
|---|---|---|
| CDN + origin shield | Reduces latency and repeated origin egress | Edge hit ratio, origin bandwidth |
| Adaptive bitrate delivery | Adjusts quality to viewer connectivity | Rebuffer ratio, start-up time |
| Multi-zone API tier | Removes single-instance dependence | Error rate, availability |
| Queue-based encoder scaling | Handles variable transcoding backlog | Queue age, processing delay |
| Cost governance | Avoids excessive network and compute spend | Cost per viewing hour |

**Validation and conclusion:**

A controlled load test should emulate increasing concurrent sessions and geographic distributions, measure playback start-up delay, rebuffer ratio, API p95 latency, CDN hit ratio, origin bandwidth and cost per viewing hour, and then test instance and zone failure. No specific buffering or throughput target is guaranteed without a workload profile. The proposed architecture separates video delivery from application control, exploits edge caching and adaptive bitrate streaming, and scales expensive components according to their actual bottlenecks. This is the most appropriate foundation for high-demand media delivery.

# QUESTION 2 – FINTECH MULTI-CLOUD CONTAINER ORCHESTRATION

Scenario: A fintech startup operates containers across multiple cloud providers but faces inconsistent environments and deployment-management challenges. The required solution must address orchestration, multi-cloud operation, service discovery, scaling, benefits and risks.

**Scenario interpretation and core issues:**

The startup needs repeatable releases, consistent security policies, predictable networking and reliable operations across providers with different infrastructure APIs and managed services. Financial workloads add stringent expectations for auditability, data protection, identity separation and resilience. Multi-cloud does not automatically deliver portability: application images may be portable while databases, load balancers, storage classes, secret stores and identity services remain provider-specific. Therefore the design should standardize what can be standardized and explicitly document cloud-specific interfaces.

**Proposed orchestration and deployment design:**

Kubernetes is recommended as the common orchestration abstraction, with one managed cluster per selected cloud and separate production and non-production environments. Application workloads should be packaged as OCI-compatible container images and stored in a trusted registry with vulnerability scanning and image signing. Infrastructure as Code, such as Terraform-compatible workflows, should provision cloud resources, while GitOps controllers reconcile Kubernetes manifests from reviewed repositories. Helm or Kustomize can define reusable deployments with small provider-specific overlays. Admission policies, namespaces, network policies, resource quotas and workload identity provide consistent governance.

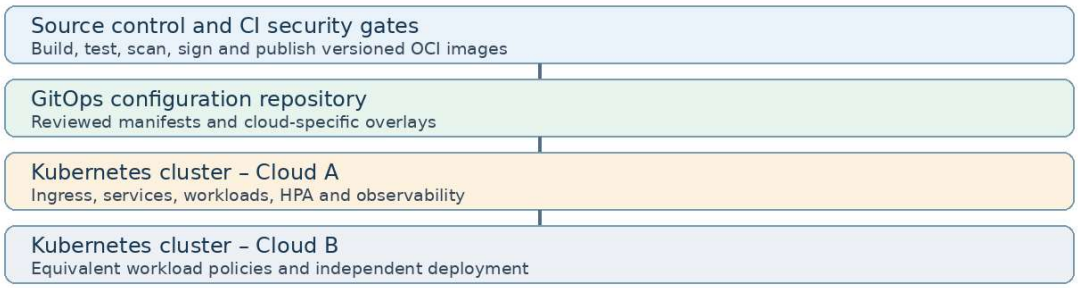



**Figure 2 – Portable application delivery across separately managed Kubernetes clusters.**

**Service discovery and network communication:**

Within each cluster, Kubernetes Services and cluster DNS expose stable service names despite changing pod addresses. Ingress controllers or gateway APIs manage north-south application traffic, while network policies restrict east-west communication.

Cross-cluster service discovery requires an explicit design: for example, regional gateways with globally resolvable service endpoints, or a multi-cluster service mesh where operational maturity justifies it. A service mesh may provide mutual TLS, traffic policies and telemetry, but it also increases cost and debugging complexity.

Avoid making every internal request cross a cloud boundary because inter-cloud latency and egress charges can become substantial.

**Efficient scaling and resilience:**

Horizontal Pod Autoscaling can increase application replicas based on CPU, memory or validated custom metrics, while cluster autoscaling adds worker nodes when pod scheduling capacity is insufficient. Queue-based services can use event-driven scaling based on backlog.

Pod disruption budgets, readiness probes, topology spread constraints and multiple availability zones protect routine maintenance and instance failures. Each cloud should be capable of serving a defined share of traffic, and traffic shifting between providers must be tested.

Stateful databases require a separate replication and consistency plan; Kubernetes alone does not make cross-cloud state highly available.

**Benefits, risks and controls:**

The benefits of multi-cloud include reduced concentration risk, flexibility to select specialized services, geographic options and potential bargaining leverage. However, the startup must account for duplicated operations, heterogeneous IAM, inconsistent network features, cross-cloud data transfer, higher observability complexity, policy drift and distributed database consistency challenges. Financial data must remain subject to applicable jurisdictional and contractual requirements.

A pragmatic starting model is active deployment in two providers with explicitly controlled data ownership and documented recovery procedures, rather than automatically adopting active-active writes across independent databases.

**Table 2. Multi-Cloud Container Deployment Risks and Mitigation Strategies**

| Risk | Impact | Mitigation |
|---|---|---|
| Configuration drift | Different security or runtime behavior | GitOps, policy-as-code, drift checks |
| Inter-cloud latency and egress | Slow requests and higher spend | Localize services and minimize cross-cloud calls |
| Secret or credential exposure | Unauthorized access to financial services | Workload identity, secret rotation, least privilege |
| Data divergence | Inconsistent balances or transaction records | Single-writer or carefully designed consensus/replication |
| Operational complexity | Slow incident response | Common telemetry, runbooks and ownership |

**Validation and conclusion:**

Acceptance testing should verify that the same signed application release deploys successfully to both clouds, services resolve correctly, autoscalers react to measured demand, network policies block unauthorized connections, and one-cluster loss can be handled according to the agreed recovery objective. Evidence should include deployment logs, policy tests, latency measurements and a failover drill. The recommended Kubernetes-plus-GitOps approach standardizes deployments without pretending that all cloud services are interchangeable, giving the fintech organization a controlled and auditable multi-cloud operating model.

# QUESTION 3 – E-LEARNING BACKUP AND DISASTER RECOVERY

Scenario: An e-learning platform has lost data after accidental deletion. The organization requires cloud-native backup, replication, versioning, RTO/RPO objectives and a continuity procedure.

**Scenario interpretation and recovery requirements:**

Accidental deletion is a logical failure, not necessarily a hardware or regional failure. A replica can immediately reproduce an accidental deletion, so replication by itself is not a backup. The platform likely contains relational learning records, object files such as assignments and videos, user identity data and configuration state. Each data class needs an appropriate retention, consistency and restoration strategy. The business should classify critical data and set approved recovery time objective (RTO), the maximum acceptable time to restore service, and recovery point objective (RPO), the maximum tolerable age of recovered data.

**Layered cloud-native protection architecture:**

Use scheduled database backups with point-in-time recovery where supported, object-storage versioning for course materials and assignments, immutable or write-protected backup copies to resist deletion and ransomware, and cross-account or cross-region backup copies for stronger separation. Infrastructure definitions, configuration, encryption-key recovery procedures and application dependencies must also be backed up. Replication may support availability during infrastructure failure, while independent versioned backups support recovery from mistaken deletion and corruption. Backup administrators should use separate least-privilege identities and retention-lock policies appropriate to the organization.

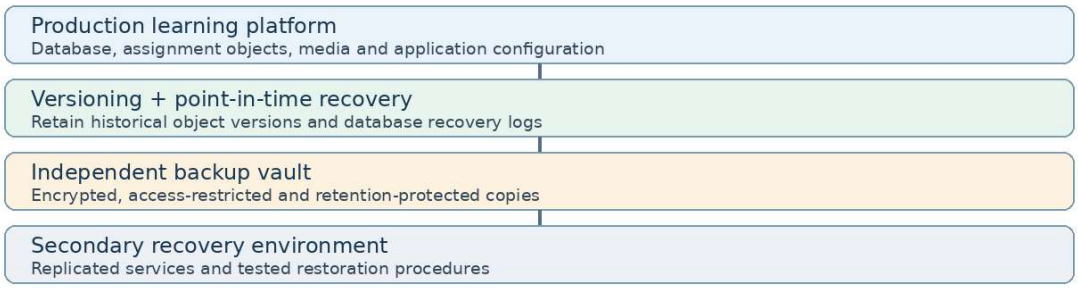



**Figure 3 – Layered protection against deletion, corruption and infrastructure failure.**

**RTO and RPO design:**

For illustration, the organization could adopt a four-hour RTO and fifteen-minute RPO for enrollment and assessment records, subject to business approval and service capabilities. The RPO requires sufficiently frequent recoverable database states, such as continuous transaction-log capture or point-in-time recovery, and monitoring of replication or log lag. The RTO requires a pretested restoration path, infrastructure readiness, credential and key availability, and operational staff who can complete recovery within the target. Static course media may tolerate different objectives from grades and submissions. These values are examples, not requirements specified in the assessment.

Suppose a deletion occurs at 14:00 and the latest verified recoverable state is 13:50. Restoring that state implies up to ten minutes of data loss for the affected dataset, within a fifteen-minute example RPO. If the application becomes fully operational by 17:00, the recovery time is three hours, within a four-hour example RTO. Such outcomes must be demonstrated through timed exercises; they cannot be assumed from product features alone.

**Business continuity and restoration procedure:**

First, detect and classify the incident using audit logs and application alerts, then restrict the offending credentials or automation to prevent repeated deletion. Second, preserve evidence and identify the precise deletion time and affected data scope. Third, select a clean recovery point and restore to an isolated environment for validation rather than overwriting the only surviving evidence. Fourth, verify record counts, referential integrity, user access and consistency between the database and stored files. Fifth, reconnect the recovered environment or selectively restore missing objects, communicate the service impact, and monitor for renewed errors. Finally, conduct a post-incident review, update access policies and rehearse the recovery runbook.

**Risks, controls and testing;**

Backup jobs can report success while individual records remain unrestorable; therefore periodic restore drills and integrity checks are essential. Other risks include insufficient retention, compromised backup credentials, missing encryption keys, inconsistent database snapshots, excessive restore time and unforeseen cloud charges. The provider should alert on failed backups, stale recovery points, replication lag, vault policy changes and unusual deletion activity. Recovery tests should document actual RTO, actual RPO, data correctness and unresolved exceptions.

**Table 3. Cloud Data Failure Scenarios and Recovery Mechanisms**

| Failure mode | Primary protection | Evidence to collect |
|---|---|---|
| Accidental file deletion | Object versioning and protected backup | Restored file version and audit event |
| Database row deletion | Point-in-time recovery | Timestamp and row-count comparison |
| Regional disruption | Secondary recovery environment | Timed failover and service checks |
| Malicious mass deletion | Immutable copy and privileged-access controls | Retention-policy test and access logs |

**Conclusion:**

A robust e-learning recovery strategy combines independent backups, versioning, protected retention, tested point-in-time recovery and replication. Clear RTO/RPO ownership and documented restoration drills transform backup from a storage feature into a verifiable business-continuity capability.

# QUESTION 4 – SECURE GOVERNMENT HYBRID CLOUD

Scenario: A government agency must retain compliance-sensitive applications on premises while using public cloud resources for less-sensitive workloads. The architecture must secure communication, identity, encryption and hybrid management.

**Scenario interpretation and data placement:**

The agency must separate data and systems according to classification, legal obligations, residency requirements and operational criticality. Sensitive citizen records, restricted databases or regulated processing may need to remain on premises; public information services, scalable portals and sanitized analytics can be hosted in the cloud when policy permits. The specific jurisdiction and compliance framework are not stated, so no particular law or certification can be presumed. The architecture must prevent a non-sensitive cloud application from becoming an uncontrolled path into sensitive on-premises systems.

**Proposed hybrid architecture and connectivity:**

A dedicated on-premises security zone hosts restricted databases and regulated applications behind internal firewalls. A cloud virtual private network hosts approved public-facing services, application gateways and permitted analytics. Private connectivity such as dedicated circuits can provide predictable enterprise networking, with encrypted VPN tunnels as an alternative or supplementary protection. A controlled integration layer exposes only necessary APIs through authenticated gateways and narrow allowlists. Network routes, firewall rules and DNS resolution should be centrally reviewed and logged. Zero-trust principles require each request to be authenticated, authorized and inspected rather than trusting traffic solely because it originates from an internal network.

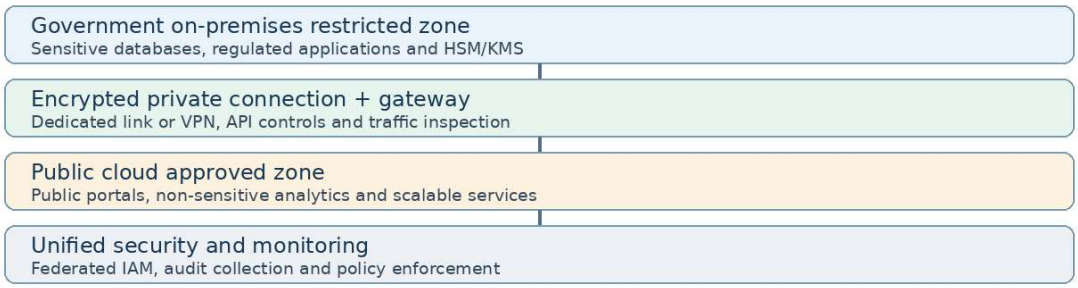



**Figure 4 – Hybrid-cloud trust boundaries and controlled communication paths.**

**Identity management and authorization:**

Use a central identity provider integrated with on-premises directories and cloud identity federation, using standards such as SAML or OpenID Connect as appropriate. Privileged administrators should use multifactor authentication, short-lived credentials, role-based or attribute-based access policies, just-in-time elevation and periodic access review. Service-to-service communication should use workload identities or managed certificates instead of long-lived embedded secrets. Cloud administrators must not automatically obtain access to sensitive on-premises databases; access should be mediated by explicitly authorized application interfaces.

**Encryption and key management:**

Protect data in transit with modern TLS and mutual TLS where appropriate for service-to-service links. Encrypt sensitive data at rest in databases, object stores, backups and endpoint storage, using centrally governed key management. For the highest-sensitivity workloads, hardware security modules and customer-controlled keys may be required by agency policy.

**Hybrid management challenges and mitigations:**

Hybrid environments create visibility gaps, heterogeneous IAM, inconsistent patch schedules, cross-boundary latency, dependency on network links, differing incident-response procedures and complex cost allocation. Data classification and workload placement must be revisited when applications change.

**Table 4. Hybrid Cloud Security Controls and Verification Methods**

| Requirement | Design control | Verification |
|---|---|---|
| Sensitive data residency | Restricted on-premises storage | Data-flow and inventory review |
| Secure communication | TLS, private routing and API gateway | Connection and firewall tests |
| Identity consistency | Federated IAM and MFA | Access review and denied-access test |
| Auditability | Centralized protected logs | End-to-end event correlation |
| Operational continuity | Redundant links and local fallback | Link-failure exercise |

**Conclusion:**

The recommended hybrid model preserves mandated on-premises controls while allowing approved public-cloud workloads to scale. Its security depends on classification-driven placement, narrow integration paths, federated least-privilege identity, encryption and consistent monitoring rather than the physical location of systems alone.

# QUESTION 5 – MULTI-REGION SAAS HIGH AVAILABILITY

Scenario: A SaaS provider must remain operational if an entire cloud region fails. The design must cover multi-region deployment, replication, failover, monitoring and proactive alerts.

**Scenario interpretation and availability objectives:**

A regional outage can remove application servers, regional networking and managed data services simultaneously. Deploying across multiple availability zones inside one region is insufficient for this scenario. The provider must maintain application capacity and recoverable state in at least two geographically separate regions, while also defining what "continue operating" means for read access, new writes and existing sessions. Availability objectives, acceptable RPO and acceptable RTO should be approved before selecting an active-passive or active-active design.

**Proposed multi-region deployment:**

A practical starting design is active-passive or active-warm deployment: Region A serves normal traffic, while Region B runs sufficient standby infrastructure to take over within the agreed recovery objective. Each region contains multiple availability zones, load-balanced stateless application instances, local caches and regional service dependencies. Global traffic routing performs health-based failover. Application images, infrastructure templates, configuration and secrets must be available in both regions. For workloads demanding near-zero interruption and supporting suitable data semantics, an active-active application tier can be considered, but multi-writer database coordination is substantially more complex.

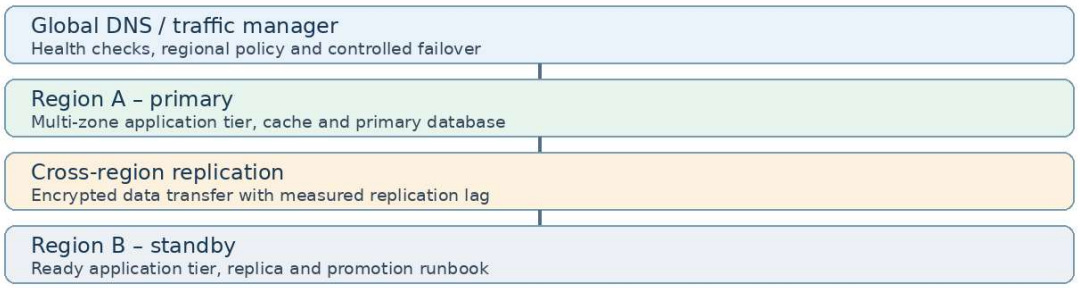



**Figure 5– Multi-region SaaS deployment with regional failover and replicated data.**

**Replication and consistency configuration:**

The database replication mode determines possible data loss. Asynchronous cross-region replication usually avoids adding remote round-trip latency to each write but may lose recent committed writes if the primary fails before replication. Synchronous replication can reduce this exposure but may raise latency and constrain availability when regions cannot communicate. A single-writer model with monitored replication lag and a well-defined promotion process is often easier to operate than uncontrolled multi-region writes. Object storage should use cross-region replication where appropriate, and queues, session stores and external integrations need explicit recovery plans. Prevent split-brain by ensuring only one writable primary is authoritative during failover.

**Failover and failback mechanisms:**

Automated health checks should distinguish a regional failure from an isolated service failure, and a failover controller must avoid switching traffic repeatedly during transient incidents. The recovery runbook should specify detection, incident declaration, database promotion, write fencing, secret and endpoint verification, traffic routing, smoke testing and stakeholder communication. DNS-based failover is influenced by resolver caching and TTL behavior; global load-balancing services may provide different traffic-switching mechanisms. When the failed region recovers, failback should be planned rather than automatic: first reconcile data, restore replication, validate integrity and only then transfer production traffic.

**Monitoring and proactive alerting:**

Monitoring should combine external synthetic checks with service-level indicators such as successful request rate, p95/p99 latency, error percentage, queue age and database availability. Infrastructure metrics should include CPU, memory, disk saturation, load-balancer health, replication lag, storage capacity and regional connectivity. Centralized logs and distributed tracing help diagnose multi-service failures, while alerts should be tied to user impact and recovery objectives. Warning thresholds should be tested to avoid both missed incidents and alert fatigue. Monitoring systems themselves should not depend exclusively on the primary region.

**Costs, risks and acceptance tests:**

Multi-region operation adds duplicate infrastructure, cross-region replication charges, storage costs and operational complexity. A warm standby can reduce expense compared with fully active-active

deployment, but may have a longer recovery time. Other risks include stale replicas, split-brain, hidden regional dependencies, failover loops and inadequate secondary capacity. Acceptance tests should deliberately disable primary-region traffic in a controlled exercise, record actual outage duration and last recoverable write, verify business transactions and restore normal operation safely. Reported RTO/RPO must be measured, not inferred from architecture diagrams.

**Table 5. Multi-Region High Availability and Fault Tolerance Controls**

| Control | Purpose | Test evidence |
|---|---|---|
| Multi-zone deployment in each region | Tolerates zone-level faults | Zone disruption test |
| Cross-region database replication | Preserves recoverable state | Lag and consistency measurements |
| Global health-based routing | Redirects clients to surviving region | Failover timestamps and request success |
| Single-writer promotion safeguards | Avoids split-brain and conflicting writes | Write-fencing and promotion logs |
| External monitoring | Detects region-wide failures | Synthetic probes and alert history |

**Conclusion:**

A two-region SaaS architecture with multi-zone application tiers, recoverable data replicas, controlled traffic failover and independent observability can maintain service through a regional outage within explicitly tested objectives. The preferred design balances recovery requirements against data-consistency risks and the real cost of duplicated capacity.